MY MEDIEVAL LIFE
Subjective Judgment & Universal Decision Engine

Exhaustive design and engineering handoff | Post–Build 16 initiative | Version 2.0 | 19 October 2026

Prepared as an implementation-ready design proposal for Claude. Not an instruction to begin integration or publish a build.

Document control

Designation	Status
Owner-established design	Requirements explicitly expressed in discussion
Proposed implementation	Engineering recommendations subject to codebase review
Open decision	Requires owner approval or documented assumption
Release gate	No implementation activation or distributable build without explicit authorization

0. Read this first: non-negotiable intent

- Every character can form a subjective opinion of every other character; the player is not special.
- An action has an event-level GOOD or BAD baseline and severity, but each observer applies a personal signed relevance/interpretation percentage.
- The observer stores nonnegative GOOD and BAD totals separately and computes NET = GOOD − BAD. NET can be fractional (e.g., −6.59).
- The net judgment is a graded likeability/disposition influence. It is NOT the decision, NOT a universal reputation, NOT an automatic romance or commerce threshold, and NOT an instruction to obey.
- Each actual decision considers context-dependent factors such as need, children, food, money, resources, alternatives, ambitions, personality, danger, relationship goals, and judgment.
- Negative judgment can be overridden by necessity; positive judgment cannot make impossible or unwanted actions happen.
- The entire design must eventually integrate across the simulation, incrementally, after a stable Build 16 equilibrium baseline is captured.

Canonical example: John judges Verbal at −6.59, but needs 50 iron and Verbal is the only supplier. John can still buy iron because scarcity and family needs outweigh dislike. A second supplier may change the outcome without changing John's opinion.

1. Scope, boundaries, and invariants

1.1 In scope

Social, economic, survival, household, romantic, political, criminal, employment, resource, cooperative, competitive, and interpersonal decisions; observed, reported, and inferred events; persistence; deterministic replay; simulation analytics.

1.2 Out of scope for initial implementation

No LLM calls, cloud AI, network dependency, scripted individual opinions, wholesale NPC scheduler replacement, universal hard-coded moral outcomes, guaranteed political/romantic behavior, or mandatory real-time all-pairs relationship calculations.

1.3 Invariants

1. World truth is separate from belief; belief is separate from moral judgment; judgment is separate from behavior.
2. Event base polarity and severity do not mutate when an observer disagrees.
3. Per-observer GOOD/BAD contributions never become negative; interpretation may reverse which ledger receives the magnitude.
4. A single belief about an event has one active contribution per observer–actor–event identity; gossip does not add duplicates.
5. Actions must be feasible before utility scoring can select them.
6. All relevant weights, thresholds, policies, and feature flags are data driven and inspectable.
7. Social systems must preserve their distinct concepts: familiarity, friendship, trust, respect, attraction, obligation, and moral judgment.
8. No action should produce an unbounded flood of morally significant events merely through repetitive animation or passive ticks.

2. Domain model and precise definitions

Term	Definition
WorldEvent	Immutable factual occurrence, with causal and contextual data.
EventSubject	Actor whose behavior is judged; events may contain multiple actors with attributed roles.
Observer	Character who knows or believes something about an event.
Belief	Observer-specific claim about event, actor, context, and confidence.
Interpretation	Signed agreement/reversal fraction in [−1,+1], relative to baseline GOOD/BAD.
Contribution	Nonnegative GOOD or BAD points attributed to one observer/subject/event belief.
JudgmentLedger	Observer→subject GOOD/BAD totals and supporting active contributions.
NetJudgment	GOOD minus BAD, a continuous disposition signal.
DecisionContext	Snapshot of needs, goals, resources, constraints, participants, and opportunities.
DecisionProfile	Configurable candidate actions, feasibility predicates, scoring factors, and policy.
Utility	Contextual desirability of an option; not automatically a probability.
OutcomeEvent	Observed consequence that may generate further beliefs/judgments.

3. Event taxonomy and objective severity

Owner intent: event begins with a GOOD or BAD classification and base severity. Proposed extension: allow operationally neutral events that affect needs, information, or relationships without posting moral points. This neutral category is an OPEN DESIGN DECISION; do not silently redefine owner's GOOD/BAD rule.

Family	Example	Typical context modifiers
Help / rescue	Help build house; defend against bandits	Cost to helper, risk, actual harm prevented
Theft / fraud	Steal iron; misrepresent goods	Victim loss, need, intent, repayment
Violence	Assault; kill; defend oneself	Aggression, defense, proportionality, outcome
Trade / contracts	Fair price; exploit scarcity; honor deal	Local norms, scarcity, deception, prior promise
Household / family	Provide food; abandon dependents	Dependent need, ability to provide
Employment	Pay wages; break agreement; rescue worker	Contract, hardship, safety
Romance / friendship	Keep confidence; betray trust; support partner	Consent, promises, relationship context
Politics / civic	Deliver promised public works; abuse office	Beneficiaries, public cost, promises
Community	Donate grain; hoard during famine	Scarcity, alternative uses, personal sacrifice
Information	Truthful warning; false accusation	Credibility, consequences, intent

Seed severities are calibration placeholders: minor help +0.5 GOOD, ordinary theft 1 BAD, bandit rescue 2 GOOD, unjustified killing 3 BAD. Severity is not equal to money amount; contextual transformations must be documented and bounded.

3.1 Event record contract

event_id: stable unique ID
simulation_tick: int; world_time: game timestamp
type_id: catalog key; baseline_polarity: GOOD | BAD [NEUTRAL pending]
base_severity: nonnegative fixed-point value
actors: [{character_id, role, attribution_share}]
victims: [character_id]; beneficiaries: [character_id]
location_id; object_ids; resource_deltas; monetary_value
context_tags: [self_defense, hunger, promise_broken, ...]
causal_parent_event_ids: []; event_group_id: optional
witness_ids: []; evidence_refs: []; provenance_version
Recommended: events are immutable. Correct mistaken facts by creating an explicit correction/evidence event or a versioned canonical fact revision, while preserving replay and auditability.

4. Subjective interpretation math

Let severity $S \geq 0$, interpretation $P \in [-1,1]$, and effective confidence $C \in [0,1]$. Define signed impact $I = S \times P \times C$. This is a proposed initial formulation, not a settled truth model. Confidence may instead be used only for belief gating; choose through tests and owner approval.

Base event	I ≥ 0	I < 0
GOOD	GOOD += I	BAD += abs(I)

BAD	BAD += I	GOOD += abs(I)

Example: Bob is robbed of materials, S=1 BAD. Bob P=+1.00 → 1 BAD. Bob's cousin P=+0.55 → 0.55 BAD. Rival John P=−0.65 → 0.65 GOOD. Unaware person has no belief/contribution, not an automatic zero-valued judgment record.

4.1 Interpreting percentages

P is signed subjective alignment with the event's baseline, not "how much the NPC witnessed," not probability of knowing, and not a negative point. Direct impact, social proximity, personal values, economic advantage, belief confidence, and personality can all influence it. Opposing forces can cancel or reverse P.

4.2 Proposed interpretable calculation

```
relevance = bounded_blend(personal_impact, social_impact,
                economic_impact, value_alignment,
                relationship_context, personality_context)
interpretation = clamp(relevance, -1.0, +1.0)
impact = severity * interpretation * confidence_policy(belief)
post_to_good_or_bad(impact, baseline_polarity)
```

The precise blend function is intentionally not fixed. A weighted average may suppress strong direct victim reactions, while unbounded addition exaggerates correlated influences. Candidate approaches: hierarchical priority for direct harm; bounded weighted sum; signed nonlinear aggregation. Compare them against a golden-case fixture suite. Expose each component in diagnostics.

4.3 Guard against circular feedback

Prior dislike can bias interpretation of ambiguous behavior, but must not automatically convert every ordinary action into escalating hostility. Cap relationship-bias effects, distinguish known harm from rumor, and avoid double applying existing friendship, family bond, and loyalty to the same influence.

5. Ledger mechanics, examples, and ambiguity

Each observer→subject ledger contains total GOOD, total BAD, NET, active contribution count, belief confidence summaries, familiarity/observation coverage, and a bounded history index. The totals remain separate even if most ordinary decisions consume NET.

Observer	GOOD	BAD	NET	Meaning
A	0	0	0	Unfamiliar/indifferent
B	50	50	0	Extensive mixed history
John	20	26.59	−6.59	Mildly unfavorable
Friend	32	15	+17	Favorable
Close ally	40	15	+25	Strongly favorable

100 actions with 50 baseline GOOD and 50 baseline BAD do not necessarily produce equal observer contributions. Most actions may be unknown, irrelevant, weakly weighted, or reversed. Therefore an observer can reasonably end at +2.65, −6.59, +17, or −30 from the same world history.

5.1 Update and correction

```
key = (observer_id, judged_actor_id, source_event_id)
```

```
old = active_contribution.get(key, ZERO)
new = evaluate_current_belief(event, observer, actor)
ledger.good += new.good - old.good
ledger.bad  += new.bad  - old.bad
active_contribution[key] = new
ledger.net = ledger.good - ledger.bad
```

Store and update contributions idempotently. Retractions subtract prior credited points from the ledger, but ledger totals themselves must not go negative. Apply fixed-point arithmetic (e.g., integer milli-points) or a carefully specified rounding policy for deterministic replay and save compatibility.

5.2 Long-term calibration

Scores can inflate as event count grows. Evaluate event spam prevention, severity caps, diminishing marginal impact for repeated identical behaviors, memory/forgiveness policies, and optional recency effects. Never add automatic decay or universal caps without explicit owner decision. Preserve lifetime historical totals if a separate effective/recent judgment is introduced.

6. Belief, knowledge, evidence, and rumor

Four separate records are recommended: factual event, communicated claim, observer belief, and observer contribution. A villager who heard an accusation can judge the accused even if the world truth differs. On correction, reverse/recompute the old contribution; do not post the original accusation again as an independent duplicate moral event.

Knowledge source	Possible effects
Direct participant	High access to consequences, but motive may remain unknown
Eyewitness	Observed action, limited knowledge of intent
Trusted testimony	Confidence depends on messenger credibility
Rumor	Uncertain actor, motive, or facts; may propagate
Fabricated claim	False belief can influence decisions until corrected
Evidence discovery	Updates actor attribution or context; triggers recomputation

Open questions: rumor travel mechanics, credibility scoring, whether believing a false allegation damages the accuser's judgment upon discovery, and whether observers remember that they were deceived. Proposed staged rollout: firsthand and direct participant events first; rumor and evidence later, with schema support from day one.

7. Universal decision architecture

The decision engine must not use a universal "judgment threshold → action" table. It should assess feasible alternatives using decision-specific considerations. A judgment of −6.59 supplies a small negative preference signal; needs and opportunities may dominate.

9. Generate available actions based on NPC needs, goals, environment, opportunities, and scheduled obligations.
10. Check hard feasibility: inventory, money/credit/barter, mobility, access, required relationships, laws, consent, time, and safety.
11. For each feasible action, assemble a typed context and select a DecisionProfile.
12. Evaluate factors independently, normalizing each to a documented common scale.
13. Apply profile weights, nonlinear conditions, personality/values, and bounded judgment influence.

14. Select an action by configured policy: best score or seeded bounded exploration; selection probability must not be confused with utility score.
15. Execute via existing systems and record actual consequences, not hypothetical intentions.
16. Publish significant outcome events and update affected beliefs/ledgers.

7.1 Proposed utility interface

```
score(action, context, npc) =
   weighted_factors(action, context, npc)
 + interaction_terms(action, context, npc)
 + judgment_influence(npc, participants, action_type)
 - contextual_risks(action, context, npc)

eligible(action) = all(hard_constraints_pass)
selected = seeded_choice(eligible_actions, selection_policy)
```

The score may be unbounded internally, but normalized factors should have explicit ranges and direction. Never interpret score=75 as “75% chance” unless the selection policy is calibrated to that meaning.

7.2 Factor registry

Factor	Examples	Type
Survival necessity	Hunger, winter fuel, dependents	High urgency / potential hard constraint
Resource position	Food, iron, wool, money, inventory	Quantity / affordability
Alternatives	Other suppliers, other friends, other jobs	Availability and opportunity cost
Time and distance	Travel, schedule conflicts, deadlines	Cost
Risk	Physical danger, theft risk, retaliation	Penalty or feasibility
Goals	Courtship, status, household, wealth	Motivation
Personal disposition	NET judgment toward relevant person	Soft preference
Trust and reliability	Debt history, contract fulfillment	Distinct from judgment
Personality and values	Generosity, pride, risk tolerance	Factor/weight modifier
Norms and law	Customs, crime consequences, consent	Constraint or penalty
Social obligations	Kinship, promises, dependents	Context-specific pressure

7.3 Worked decision: the 50 iron problem

John needs 50 iron; Verbal has 50 iron; no other supplier; John judges Verbal at −6.59. The figures below are deliberately illustrative, not approved balance values.

Factor	Score (0–100)	Weight	Contribution
Iron necessity	90	30%	27.0
No alternatives	100	25%	25.0
Affordability	70	15%	10.5
Urgency	75	20%	15.0
Disposition mapped from −6.59	43.41	10%	4.341
Total		100%	81.841

A high illustrative willingness score does not guarantee purchase: price, stock, credit, and merchant consent must pass feasibility. If a second supplier appears, the alternatives factor and candidate set change; judgment stays −6.59. If John's children are hungry and iron is needed to earn wages, survival-related considerations may be much stronger. If he can wait, dislike may prevail.

7.4 Opposing motivations

NPCs may behave inconsistently across situations without violating internal logic. A merchant can despise a customer but sell to feed a family. A spouse can love a partner but refuse to cover up a murder. A candidate can be personally disliked but supported for a specific village policy. Do not automatically update judgment simply because an NPC chooses to cooperate.

8. Integration inventory: every major domain

System	Decision examples	Judgment role	Other dominant factors
Commerce	Buy/sell, prices, credit, trade partner	Preference/discount/refusal pressure	Need, stock, price, cash, alternatives
Survival	Share food, seek aid, protect family	Recipient priority	Hunger, dependents, supply
Production	Exchange inputs, lend tools	Cooperation bias	Work orders, shortages, cost
Employment	Hire, fire, promote, wage offers	Preference/recommendation	Skill, wages, staffing
Households	Support kin, allocate supplies	Interpersonal preference	Dependents, budget, obligations
Friendship	Initiate visits, favors, defend	Likeability	Familiarity, shared interests, time
Courtship	Approach, date, accept/reject	Soft compatibility signal	Attraction, consent, availability
Marriage	Propose, accept, remain together	Relationship influence	Mutual goals, readiness, attraction
Conflict	Retaliate, forgive, avoid	Grievance pressure	Risk, law, strength, temperament
Crime	Steal, report, conceal, testify	Target/ally bias	Need, opportunity, ethics, fear
Information	Share truth, gossip, lie	Trust and bias	Evidence, loyalty, risk
Civic/politics	Support candidates, vote, help projects	Candidate disposition	Policy, expected outcomes, interests
Security	Defend or abandon someone	Ally preference	Threat, duty, capability
Resource access	Share wool/iron/grain	Cooperation bias	Scarcity, tradeoffs, promises
Social networks	Introduce, invite, exclude	Affinity	Availability, norms, status

This is an integration inventory, not a command to wire every domain immediately. Claude must map it to actual code paths and identify all existing decision entry points. Each integration needs its own tests and feature flag.

9. Relationships, romance, and politics: preserve distinctions

Existing friendship, familiarity, trust, romance, gifts, and courtship pacing remain distinct. Positive judgment can increase willingness to engage, but cannot create attraction or consent. Existing multi-day contact requirements

must not be bypassed by a single high-scoring event. Political preferences may conflict with personal liking; support must evaluate candidate goals and village interests as well as judgment.

10. Personality, values, needs, and goals

NPCs should differ in factor weights and thresholds, not just in cosmetic dialogue. Trait profiles must be bounded, interpretable, and consistent. A proud NPC may refuse an unpleasant supplier when alternatives exist; a pragmatic NPC may prioritize essential supplies; a generous NPC may share excess food; a fearful NPC may refuse risky assistance despite affection.

Recommended hierarchy: immutable/slow traits → medium-term priorities and social ties → current physiological/economic needs → decision-specific opportunities → selection. Ensure needs are continuously updated by existing simulation mechanics and are not fabricated for scoring convenience.

11. Time, consequences, and feedback loops

Every selected action can change resources, schedules, social states, and future options. Only meaningful consequences generate judgment events. Example: John buys iron despite dislike → iron and money change hands → if Verbal fulfills the agreement fairly, a small positive event may be perceived → John's NET becomes slightly less negative → later trade becomes somewhat easier. If Verbal cheats him, the opposite occurs.

Avoid unstable positive feedback: disliking someone must not automatically generate a BAD event on every avoidance tick; a single refusal should not recursively trigger infinite retaliation. Add causal chains, event deduplication, cooldowns, and safeguards for action→event→decision loops.

12. Godot-oriented component contracts

These are candidate services; reuse existing Godot managers where possible. Inspect the repository before selecting node ownership, resource types, autoloads, signals, and persistence.

Component	Responsibility	Primary interface
EventCatalog	Definitions and context modifiers	get_definition(type_id)
WorldEventBus	Emit immutable simulation events	publish(event)
KnowledgeStore	Observer claims and confidence	learn_or_update(observer, claim)
JudgmentEvaluator	Observer-specific signed interpretation	evaluate(event, belief, observer)
JudgmentLedgerStore	Idempotent contributions and totals	replace_contribution(key, value)
SocialGraph	Kinship, familiarity, friendship, trust	get_links(a,b)
DecisionContextBuilder	Assemble current state snapshot	build(npc, situation)
ActionProvider	Generate feasible candidates	enumerate_actions(context)
DecisionProfileRegistry	Factor configuration by action family	get_profile(action_type)
UtilityEvaluator	Score and explain candidates	evaluate(action, context)
ActionExecutor	Use existing gameplay actions	execute(selected_action)
SocialDiagnostics	Inspect events, decisions, ledgers	trace(event_or_decision_id)

12.1 Pseudocode: event processing

```
on_world_event(event):
  validate_event(event)
  store_fact(event)
  for observer in discover_eligible_observers(event):
```

```
    claim = make_initial_belief(event, observer)
    knowledge_store.upsert(claim)
    for actor in attributed_actors(claim):
      old = ledger.get_contribution(observer, actor, event.id)
      new = evaluator.evaluate(event, claim, observer, actor)
      ledger.replace_contribution(old, new)
    diagnostics.record_interpretation(event, observer)
```

12.2 Pseudocode: decision processing

```
decide(npc, situation):
  ctx = context_builder.build(npc, situation)
  candidates = action_provider.enumerate(ctx)
  feasible = [a for a in candidates if constraints_pass(a, ctx)]
  if feasible.is_empty(): return fallback_idle_or_replan(ctx)
  scores = [utility.evaluate(a, ctx, judgment_store) for a in feasible]
  chosen = seeded_selection(scores, ctx.seed_stream)
  diagnostics.record(ctx, scores, chosen)
  return executor.execute(chosen)
```

12.3 Data versioning and saves

Persist event IDs, active beliefs, contribution snapshots, GOOD/BAD ledgers, deterministic RNG state/streams, and schema version. Implement migrations and corruption checks. Keep separate world-truth and subjective belief stores. Avoid expensive reconstruction from the entire world event log on every load; provide an offline ledger-rebuild validator for QA.

13. Performance, scale, and deterministic replay

- Event-driven updates, not every-NPC-to-every-NPC scoring each frame.
- Sparse observer→subject ledgers, allocated when relevant.
- Candidate actions limited to plausible, nearby, socially connected, or goal-relevant choices.
- Cache net judgment; invalidate on contribution changes.
- Use budgeted processing for rumor propagation and distant NPCs.
- Maintain deterministic ordering for simultaneous events and actor/observer iteration.
- Use fixed-point arithmetic or explicitly deterministic rounding.
- Separate simulation ticks from frame-rate-dependent animation.
- Benchmark population sizes and save/load duration against Build 16 baseline before setting hard budgets.

Recommended metrics: judgments processed per simulated day, active ledgers, average event fanout, decision evaluations per day, peak frame time, simulation days per wall-clock minute, memory footprint, save size, and deterministic hash comparison.

14. Observability and debugging

Provide developer-only screens or logs that answer: “What did John believe?”, “Why was the theft rated 0.55 BAD?”, “Why did John buy iron despite −6.59?”, and “Which factor caused this decision to change?”

- Judgment inspector: observer, subject, GOOD, BAD, NET, familiarity, key events, confidence, revisions.
- Decision trace: candidates, feasibility failures, normalized factors, weights, interactions, utility, selected action, seed.
- Event explorer: canonical truth, actor attribution, witnesses, propagated claims, causal chain.
- Village dashboard: judgment distributions, social connectivity, trade refusals, family formation, shortages, employment, crime, cooperation.

- Comparative diagnostics: matched-seed judgment OFF vs ON, with differences by subsystem and causal attribution where supportable.

15. Test fixtures and expected results

Test	Input	Expected
Victim theft	BAD 1; P=+1; C=1	BAD +1
Cousin reaction	BAD 1; P=+0.55; C=1	BAD +0.55
Rival reversal	BAD 1; P=−0.65; C=1	GOOD +0.65
Disapproval of GOOD	GOOD 2; P=−0.5; C=1	BAD +1
Indifference	BAD 3; P=0	No contribution
Net example	GOOD 20; BAD 26.59	NET −6.59
Rumor repeat	Same belief delivered twice	No duplicate contribution
Belief correction	Replace actor attribution	Old ledger reversed, new actor updated
Scarcity trade	NET −6.59; sole iron supplier	Purchase remains a feasible, potentially preferred choice
Marriage boundary	High NET; no attraction/consent	No automatic marriage
Determinism	Same seed/state/actions	Same decision/event/ledger hashes

15.1 Integration regression suites

Run unit, property, scenario, save/load, replay, stress, and soak tests. Check conservation of money/resources during transactions; no impossible purchases; no duplicate moral postings; no runaway score inflation; no infinite action loops; no courtship pacing regression; no spontaneous family formation due solely to high NET.

16. Equilibrium experiment design

Treat Build 16 as the stable control, not as a target to preserve at all costs after enabling social behavior. Capture baseline seeds, initial population, households, food stocks, housing, employment, prices, resource flows, births/deaths, travel times, and event rates. Compare 100-day and 365-day runs with identical initial states and RNG conventions. Where changed event order necessarily diverges RNG, use subsystem-specific deterministic streams and report the limitation.

Metric	Why it matters
Population / births / deaths	Social dynamics may affect family formation and survival
Food / fuel reserves	Cooperation or trade avoidance may produce shortages
Housing occupancy	Relationships affect household formation
Employment and wages	Trust and judgment may alter labor markets
Production throughput	Trade refusal can interrupt input supply
Prices and liquidity	Preference-based trade can distort markets
Marriage / courtship	Relationship effects must remain plausible
Conflict / crime	Grudges and misinformation may cascade
Social graph fragmentation	Isolated NPCs may become trapped
Performance / save size	New state must remain tractable

Run ablations: judgment OFF; judgment ON with only trade influence; judgment ON with only social influence; full enabled. Investigate root causes rather than masking instability through global bonuses. Expect the new engine to change equilibrium; measure whether resulting adaptations are coherent.

17. Staged execution plan and approval gates

17. Gate A — Build 16 stabilization: validate baseline, preserve seeds and remote commit, inventory known defects. No SJDE implementation yet.
18. Gate B — Architecture audit: map actual Godot services, existing relationships, economic decisions, event hooks, save format, and risk. Deliver a proposed integration map and open questions.
19. Gate C — Isolated prototype: event catalog, belief record, signed interpretation, ledger, tests. No impact on live village behavior.
20. Gate D — Decision prototype: reusable factor registry, utility profiles, iron scenario, explanation traces, deterministic tests.
21. Gate E — First gated integration: one low-risk decision domain, feature flag OFF by default, matched-seed tests.
22. Gate F — Incremental rollout: trade, aid, friendship, courtship, employment, crime, politics, etc., as existing systems permit.
23. Gate G — Full soak and calibration: 100-/365-day simulation comparison, failure analysis, performance and save migration.
24. Gate H — Owner review: approve any release/build and broad enablement separately.

Git workflow: use remote GitHub staging/WIP branch for approved work; commit and push meaningful milestones; do not treat local or cloud-only branches as durable. “Do not build/release” does not prohibit coding, testing, review, or remote preservation once development is authorized.

18. Risk register and mitigations

Risk	Mitigation
Social score inflation	Meaningful event filter; calibration; repeated-event policies
Duplicate gossip points	Stable contribution keys; replace rather than append
Feedback spirals	Cooldowns, causal event groups, bounded modifiers
Unrealistic NPC compliance	Hard feasibility, consent, separate attraction/trust
Trade economy collapse	Control runs, gradual integration, supplier alternatives
Performance explosion	Sparse data, event-driven updates, tiered simulation
Save incompatibility	Versioned schemas and migration tests
Opaque behavior	Decision traces and factor attribution
Excessive moral simplification	Context tags, reversible subjective interpretations
Accidental scope creep	Explicit owner-established vs proposed vs open decisions

19. Owner decisions still open

25. Should there be a NEUTRAL objective event type, or must every morally judged event begin GOOD/BAD?
26. Should scores decay or only change through new events, forgiveness, and corrected beliefs?
27. Should different NPCs have different disposition thresholds, or one shared calibrated scale?
28. Should very large GOOD/BAD totals with NET≈0 create special ambivalence behavior?
29. How much may prior judgment bias interpretation of ambiguous events?

30. How should confidence and rumors influence the points: multiplier, threshold, or separate belief layer?
31. How far should gossip spread, and should its reach depend on social proximity and personality?
32. How should legal guilt, moral judgment, and personal loyalty interact?
33. Should NPCs always choose highest utility or sometimes choose near-optimal options via seeded randomness?
34. What judgment information, if any, can the player see?
35. How should memory persistence and forgiveness vary by event severity?
36. What constitutes an event important enough to score?
37. How should individual decision weights be learned, configured, or tuned?
38. Which domain should receive the first live integration after the isolated prototype?

Claude should first recommend defaults, identify implementation consequences, and group questions by urgency. Do not interrupt for trivial details that can remain configurable; do ask before committing to irreversible gameplay semantics.

20. Acceptance checklist

- Every NPC can judge every other NPC using the same architecture.
- Different observers can assign opposite signs to one event.
- Fractional weighted contributions and net scores are mathematically correct.
- GOOD/BAD histories are preserved separately and corrections are idempotent.
- Net judgment is only one contextual decision factor.
- NPCs may trade or cooperate despite negative judgment when circumstances justify it.
- Positive judgment never overrides feasibility, attraction, or consent.
- Existing social, economic, household, and scheduling systems retain ownership of their behavior.
- All relevant decisions can be inspected and explained.
- Simulation remains reproducible, saveable, and performant.
- Build 16 baseline is protected; activation and releases are explicitly gated.

21. Final instruction to Claude

Do not interpret this as a request to immediately implement the feature. First confirm Build 16 baseline readiness and audit the real codebase. Return: (1) current architecture map, (2) proposed service/data contracts and integration points, (3) risks and migration plan, (4) owner questions with recommended defaults, (5) phased implementation plan with tests and performance budgets, and (6) explicit confirmation that no build or release will occur without approval. Once authorized, implement incrementally on remote Git staging/WIP, keep existing approved work moving, and never replace the owner's subjective judgment-as-one-factor principle with a simple reputation threshold system.

Design maxim: The event is what happened. Belief is what someone thinks happened. Judgment is what that means to them. Decision is what they choose to do given everything else in their life.